

Real-Time 3D Hand Gesture Recognition

Using MediaPipe and PyTorch

Soheil Hemmat

Contents

1	Introduction	2
1.1	Project Objectives	3
1.2	System Overview	3
2	Methodology	4
2.1	Hand Detection Using MediaPipe	4
2.2	Feature Extraction	5
2.3	Gesture Classification Model	6
3	Dataset and Training	7
3.1	Dataset Preparation	7
3.2	Model Training	7
3.3	Model Evaluation	8
4	Real-Time Application	9
5	Discussion	10
6	Limitations	11
7	Future Work	11
8	Conclusion	12

Abstract

Human-computer interaction (HCI) aims to create more natural and efficient ways for users to communicate with computer systems. Hand gesture recognition is an important approach in this field because it enables interaction with digital systems through natural movements without requiring traditional input devices such as keyboards, mice, or physical controllers. In this project, a real-time 3D hand gesture recognition system was developed using computer vision and deep learning techniques.

The proposed system uses a standard webcam as the input device. The captured video frames are processed using OpenCV and MediaPipe Hands [3, 1] to detect the hand structure and extract three-dimensional landmarks. The extracted landmark coordinates are converted into numerical feature vectors containing hand position information and additional distance-based features. These features are then provided to a lightweight PyTorch neural network for gesture classification.

The developed classifier recognizes three predefined gestures: Open Hand, Fist, and Thumbs Up. A Streamlit-based graphical interface was implemented to display the camera feed, detected gesture, confidence score, and corresponding control command in real time. The evaluation results show that the proposed system achieves approximately 93% accuracy on test data, demonstrating that MediaPipe landmark extraction combined with a neural network classifier provides an effective solution for real-time hand gesture recognition.

Chapter 1

Introduction

Human-computer interaction has become an important area of computer science with the goal of improving communication between humans and computer systems. Traditional interaction methods such as keyboards and mice are increasingly being complemented by more natural approaches, including speech recognition, body tracking, and hand gesture recognition.

Hand gesture recognition provides an intuitive method for interacting with computer systems without physical contact. This technology has applications in different areas such as virtual reality systems, smart environments, assistive technologies, and interactive

control systems. By analyzing human hand movements, computer systems can interpret user commands and provide appropriate responses.

The objective of this project is to develop a real-time hand gesture recognition system using computer vision and deep learning methods. The system receives video input from a webcam, detects the user's hand, extracts useful features from hand landmarks, and classifies the performed gesture using a neural network model.

The proposed system combines three main technologies. MediaPipe Hands is used for extracting three-dimensional hand landmarks [1], PyTorch is used for gesture classification [2], and Streamlit is used to create a real-time visualization interface [4]. The system recognizes three gesture classes: Open Hand, Fist, and Thumbs Up, where each gesture is mapped to a specific control command.

1.1 Project Objectives

The main objective of this project is to design and implement a complete real-time hand gesture recognition system capable of detecting and classifying hand movements using computer vision and deep learning methods.

The system focuses on extracting meaningful information from hand movements, reducing the complexity of input data, and achieving real-time gesture classification using a lightweight neural network model.

The main objectives of this project are:

- Develop a real-time hand detection system using webcam input.
- Extract three-dimensional hand information using MediaPipe Hands.
- Convert hand landmarks into numerical feature vectors.
- Design and train a neural network classifier for gesture recognition.
- Develop a graphical interface for displaying recognition results.
- Evaluate the performance of the trained model.

1.2 System Overview

The developed system follows a complete processing pipeline that starts from camera acquisition and continues through feature extraction, gesture classification, and result visualization.

The webcam captures video frames containing the user's hand movements. The captured frames are processed using OpenCV [3] and provided to MediaPipe Hands [1] for hand detection and landmark extraction. The extracted hand information is then converted into numerical features and used as input for the PyTorch neural network classifier.

The classifier predicts the corresponding gesture category based on the extracted features. The predicted gesture, confidence score, and associated command are displayed through the Streamlit interface.

The overall architecture of the proposed hand gesture recognition system is shown in Figure 1.1.

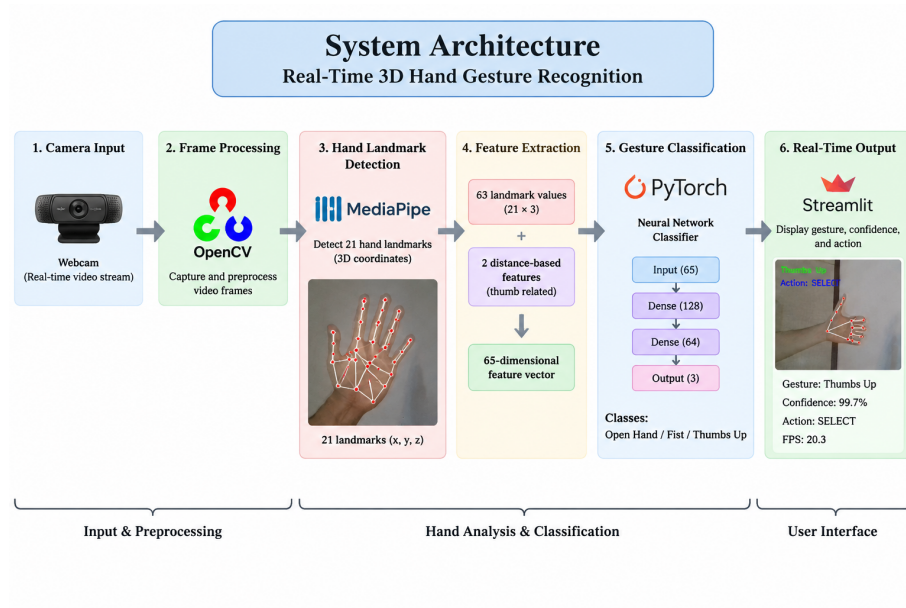


Figure 1.1: Overall architecture of the hand gesture recognition system

The main processing stages are:

1. Capturing video frames from the webcam.
2. Detecting hand landmarks using MediaPipe Hands.
3. Extracting numerical features from the detected hand.
4. Classifying the gesture using the PyTorch neural network.
5. Displaying the recognition result through the Streamlit interface.

Chapter 2

Methodology

2.1 Hand Detection Using MediaPipe

The hand detection component of the system is implemented using MediaPipe Hands, a real-time hand tracking framework developed by Google. The main purpose of this component is to detect the hand structure from camera frames and extract landmark information required for gesture recognition.

Instead of processing raw images directly, the system uses hand landmarks as a compact representation of the hand structure. This reduces the amount of input data and allows the

classification model to operate with lower computational cost while maintaining important spatial information.

MediaPipe Hands [1] detects 21 landmarks on the human hand. Each landmark contains three-dimensional coordinates:

$$(x, y, z)$$

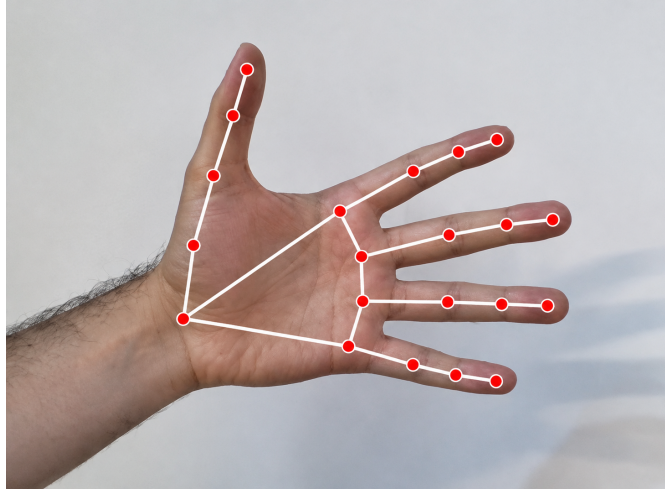


Figure 2.1: Example of 21 hand landmarks extracted using MediaPipe Hands where:

- x represents the horizontal position of the landmark.
- y represents the vertical position of the landmark.
- z represents the relative depth information.

These three-dimensional coordinates provide information about the hand geometry and are used as the basis for feature extraction and gesture classification.

2.2 Feature Extraction

Before providing the data to the neural network, the detected hand landmarks are converted into numerical features.

MediaPipe Hands provides 21 landmarks, and each landmark contains three coordinates. Therefore, the initial feature representation contains:

$$21 \times 3 = 63$$

features representing the three-dimensional position of the hand landmarks.

To improve the representation of the thumb position, two additional distance-based features are calculated:

- Distance between thumb tip and thumb base.
- Distance between thumb tip and thumb joint.

The Euclidean distance between two three-dimensional points is calculated as:

$$d = \sqrt{(x_2 - x_1)^2 + (y_2 - y_1)^2 + (z_2 - z_1)^2}$$

After adding these two distance values, the final feature vector size becomes:

$$63 + 2 = 65$$

Therefore, each detected hand is represented by a 65-dimensional feature vector, which is used as the input of the neural network classifier.

2.3 Gesture Classification Model

After feature extraction, the extracted features are classified using a lightweight fully connected neural network implemented in PyTorch [2].

The network architecture is:

- Input layer: 65 features
- Fully connected layer: 128 neurons
- ReLU activation
- Fully connected layer: 64 neurons
- ReLU activation
- Output layer: 3 classes

The output classes are Open Hand, Fist, and Thumbs Up.

The network structure is:

$$65 \rightarrow 128 \rightarrow 64 \rightarrow 3$$

During prediction, Softmax converts the output into class probabilities, and the highest probability class is selected.

Chapter 3

Dataset and Training

3.1 Dataset Preparation

Before training the classification model, the hand gesture data was processed and converted into a suitable numerical representation.

The raw hand samples were not directly used as neural network input. Instead, each sample was processed using MediaPipe Hands to extract three-dimensional hand landmarks. For each detected hand, MediaPipe provides 21 landmarks, and each landmark contains three coordinates (x, y, and z). Therefore, the initial feature representation contains:

$$21 \times 3 = 63$$

features representing the three-dimensional position of the hand landmarks.

To provide additional information about the thumb position and hand geometry, two extra distance-based features were calculated:

- Distance between thumb tip and thumb base.
- Distance between thumb tip and thumb joint.

The Euclidean distance between two three-dimensional points is calculated as:

$$d = \sqrt{(x_2 - x_1)^2 + (y_2 - y_1)^2 + (z_2 - z_1)^2}$$

After adding these two values, the final feature vector size becomes:

$$63 + 2 = 65$$

Each hand sample was therefore represented by a 65-dimensional feature vector, which was used as the input of the PyTorch classification model. The processed data was divided into training and validation sets for model training and evaluation, and the generated feature vectors were used for training the gesture classifier.

3.2 Model Training

After preparing the feature dataset, the neural network was trained using the extracted 65-dimensional feature vectors.

The model was designed as a lightweight fully connected neural network. It learns the relationship between the extracted hand features and their corresponding gesture classes.

Since the input data already contains meaningful information from MediaPipe landmarks, a small network architecture is sufficient for this classification task.

During the training process, Cross Entropy Loss was used as the loss function for the classification problem. The optimizer updated the network weights based on the prediction error between the predicted class and the actual gesture label.

After completing the training process, the model was evaluated using unseen validation samples. The classifier achieved an accuracy of approximately 93%, showing that the extracted hand landmark features contain sufficient information for recognizing the selected gesture classes.

3.3 Model Evaluation

The performance of the trained model was evaluated using a confusion matrix and classification report.

The confusion matrix provides information about the number of correctly and incorrectly classified samples for each gesture class.

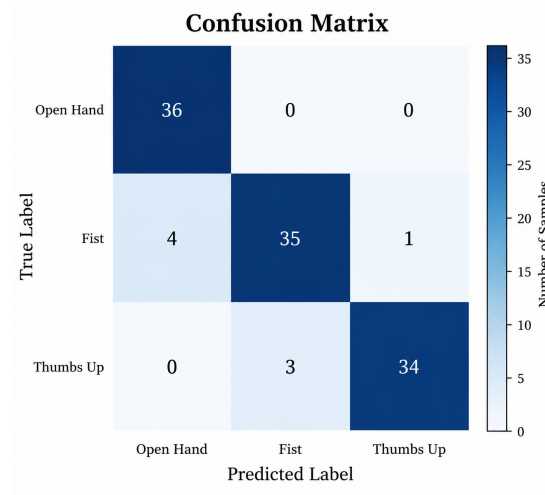


Figure 3.1: Confusion Matrix of the trained gesture classification model

The classification report provides precision, recall, and F1-score values for each gesture class.

Classification Report				
Class	precision	recall	f1-score	support
Open Hand	0.90	1.00	0.95	36
Fist	0.92	0.88	0.90	40
Thumbs Up	0.97	0.92	0.94	37
accuracy	-	-	0.93	113
macro avg	0.93	0.93	0.93	113
weighted avg	0.93	0.93	0.93	113

Figure 3.2: Classification Report of the trained model

The results show balanced performance across the three gesture classes: Open Hand, Fist, and Thumbs Up. The obtained accuracy demonstrates that the selected feature extraction method and neural network architecture are effective for real-time gesture recognition.

Chapter 4

Real-Time Application

The final system was implemented as a real-time application using Streamlit [\[4\]](#). The purpose of this interface is to provide a simple and interactive way to observe the gesture recognition process and display the prediction results.

The application receives continuous frames from the webcam and processes them through the complete recognition pipeline. Each frame is analyzed to detect the hand, extract landmarks, generate the feature vector, and predict the corresponding gesture class using the trained neural network model.

The interface provides real-time information about the recognition process. The user can observe the live camera feed, detected hand landmarks, predicted gesture class, confidence score, corresponding control command, and processing speed.

The main components displayed in the interface are:

- Live webcam feed.
- Detected hand landmarks.
- Predicted gesture class.
- Confidence score of the prediction.
- Corresponding action command.
- Processing speed (FPS).

The final application interface is shown in [Figure 4.1](#).

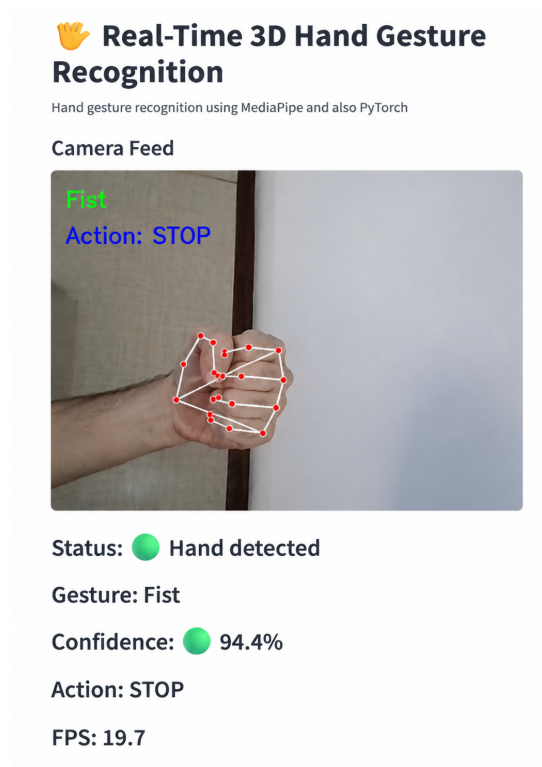


Figure 4.1: Real-time gesture recognition interface

The developed application is capable of continuously processing webcam frames and providing gesture recognition results in real time. The combination of MediaPipe for hand tracking, PyTorch for classification, and Streamlit for visualization creates a complete interactive system for human-computer interaction.

Chapter 5

Discussion

The developed system demonstrates that combining computer vision techniques with deep learning methods can provide an effective solution for real-time hand gesture recognition. The use of hand landmarks instead of raw images reduces input complexity and allows efficient operation using a standard webcam.

The achieved accuracy of approximately 93% shows that the selected feature extraction method and lightweight neural network architecture are suitable for recognizing the selected gesture classes. The landmark-based approach provides a compact representation of the hand structure while maintaining important spatial information required for classification.

The implemented Streamlit interface also improves the usability of the system by

providing real-time visualization of the camera feed, detected gesture, confidence score, and corresponding command.

Chapter 6

Limitations

Although the developed system provides good performance, several limitations exist.

- The current model supports only three gesture classes.
- Recognition performance may decrease under different lighting conditions.
- The system was mainly tested using a single hand.
- Large changes in hand orientation or distance from the camera may affect detection accuracy.

Chapter 7

Future Work

Although the developed system provides real-time gesture recognition, several improvements can be considered for future development.

Possible future improvements include:

- Adding more gesture classes to increase system flexibility.
- Collecting a larger and more diverse dataset.
- Improving robustness under different lighting conditions and backgrounds.
- Using temporal information from multiple frames to improve prediction stability.

Chapter 8

Conclusion

In this project, a real-time 3D hand gesture recognition system was developed using MediaPipe, PyTorch, OpenCV, and Streamlit.

The system detects hand landmarks from webcam input, extracts a 65-dimensional feature vector, and uses a neural network classifier to recognize different gesture categories. The final application successfully recognizes three gestures: Open Hand, Fist, and Thumbs Up.

The evaluation results show that the trained model achieves an accuracy of approximately 93%, demonstrating that the combination of MediaPipe landmark extraction and a lightweight neural network provides an effective approach for real-time gesture recognition.

This project shows how computer vision and deep learning techniques can be combined to create a practical human-computer interaction system using standard hardware.

Bibliography

- [1] F. Zhang, V. Bazarevsky, A. Vakunov, A. Tkachenka, G. Sung, C. Chang, and M. Grundmann, “MediaPipe Hands: On-device Real-time Hand Tracking,” Google Research, 2020.
- [2] A. Paszke et al., “PyTorch: An Imperative Style, High-Performance Deep Learning Library,” Advances in Neural Information Processing Systems, 2019.
- [3] G. Bradski, “The OpenCV Library,” Dr. Dobb’s Journal of Software Tools, 2000.
- [4] Streamlit Documentation, “Streamlit: The fastest way to build and share data apps.”